JAIDEV EDUCATION SOCIETY'S
JD COLLEGE OF ENGINEERING AND MANAGEMENT
KATOL ROAD, NAGPUR

Name :- Prof. Vanshika Haribhau Khapekar
Subject :- Operating System

Unit III

Scheduling concepts, scheduling algorithms, algorithm evaluation, multiple processor scheduling, real time scheduling.

Scheduling Concepts

Scheduling is the process of arranging, controlling and optimizing work and workloads in a production process or manufacturing process. Scheduling is used to allocate plant and machinery resources, plan human resources, plan production processes and purchase materials.

Operating System Scheduling algorithms

A Process Scheduler schedules different processes to be assigned to the CPU based on particular scheduling algorithms. There are six popular process scheduling algorithms which we are going to discuss in this chapter −

- First-Come, First-Served (FCFS) Scheduling
- Shortest-Job-Next (SJN) Scheduling
- Priority Scheduling
- Shortest Remaining Time
- Round Robin(RR) Scheduling
- Multiple-Level Queues Scheduling

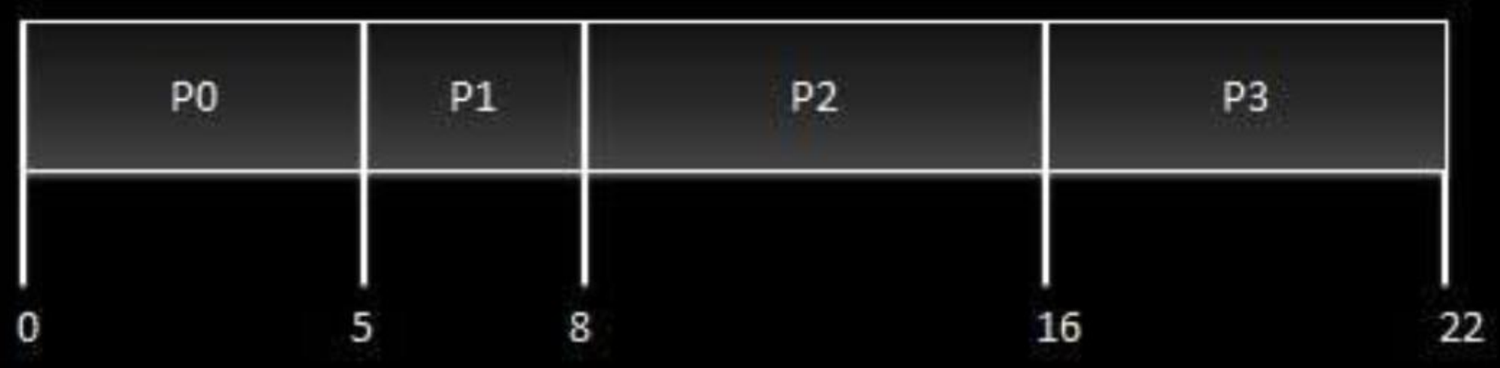
These algorithms are either **non-preemptive or preemptive**. Non-preemptive algorithms are designed so that once a process enters the running state, it cannot be preempted until it completes its allotted time, whereas the preemptive scheduling is based on priority where a scheduler may preempt a low priority running process anytime when a high priority process enters into a ready state.

First Come First Serve (FCFS)

- Jobs are executed on first come, first serve basis.

- It is a non-preemptive, pre-emptive scheduling algorithm.
- Easy to understand and implement.
- Its implementation is based on FIFO queue.
- Poor in performance as average wait time is high.

Process	Arrival Time	Execute Time	Service Time
P0	0	5	0
P1	1	3	5
P2	2	8	8
P3	3	6	16



Wait time of each process is as follows −

Process	Wait Time : Service Time - Arrival Time
P0	0 - 0 = 0
P1	5 - 1 = 4
P2	8 - 2 = 6
P3	16 - 3 = 13

Average Wait Time: (0+4+6+13) / 4 = 5.75

Examples

1. Process ID Process Name Arrival Time Burst Time
2. _
3. P 1 A 0 6
4. P 2 B 2 2
5. P 3 C 3 1
6. P 4 D 4 9
7. P 5 E 5 8

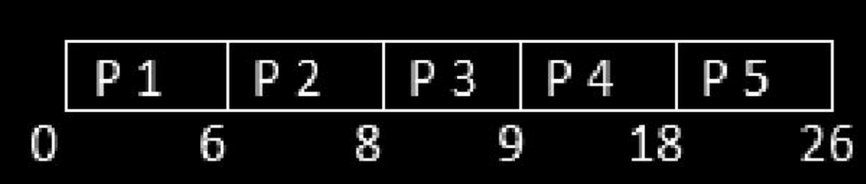
Now, we are going to apply FCFS (First Come First Serve) CPU Scheduling Algorithm for the above problem.

In FCFS, there is an exception that Arrival Times are not removed from the Completion Time.

Here, in First Come First Serve the basic formulas do not work. Only the basic formulas work.

Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
P 1	0	6	6	6	0
P 2	2	2	8	8	6
P 3	3	1	9	9	8
P4	4	9	18	18	9
P 5	5	8	26	26	18

Gantt Chart



Average Completion Time = The Total Sum of Completion Times which is divided by the total number of processes is known as Average Completion Time.

Average Completion Time =(CT1 + CT2 + CT3 + ... + CTn) / n

Average Completion Time

1. Average CT = (6 + 8 +9 + 18 + 26) / 5
2. Average CT = 67 / 5
3. Average CT = 13. 4

Average Turn Around Time = The Total Sum of Turn Around Times which is divided by the total number of processes is known as Average Turn Around Time.

Average Turn Around Time = (TAT1 + TAT2 + TAT3 + ... + TATn) / n

Average Turn Around Time

1. Average TAT = (6 + 8 + 9 +18 +26) / 5

2. Average TAT = 13.4

Average Waiting Time = The Total Sum of Waiting Times which is divided by the total number of processes is known as Average Waiting Time.

Average Waiting Time = (WT1 + WT2 + WT3 + ... + WTn) / n

Average Waiting Time

1. Average WT = (0 + 6 + 8 + 9 + 18) / 5
2. Average WT = 41 / 5
3. Average WT = 8.2

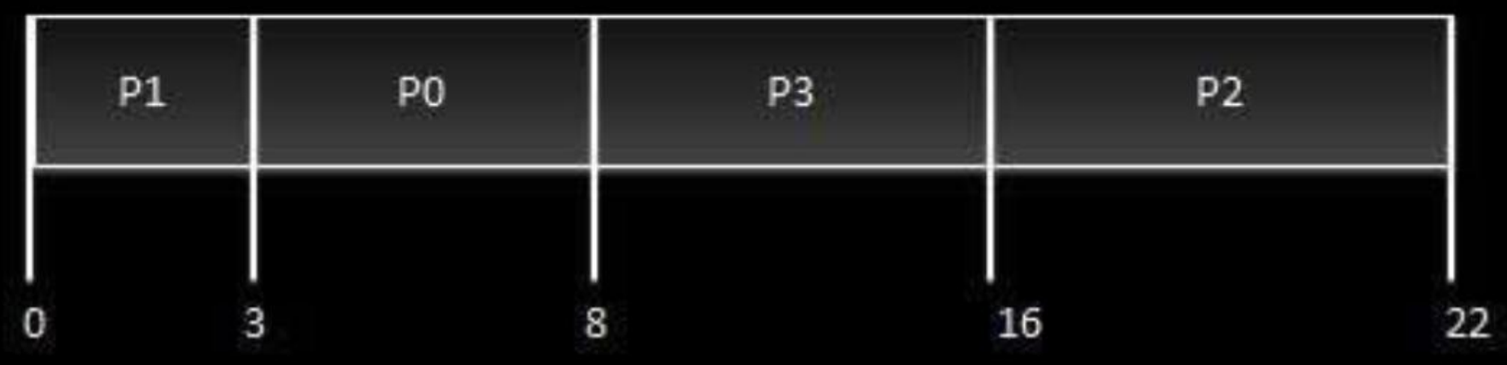
Shortest Job Next (SJN)

- This is also known as **shortest job first**, or SJF
- This is a non-preemptive, pre-emptive scheduling algorithm.
- Best approach to minimize waiting time.
- Easy to implement in Batch systems where required CPU time is known in advance.
- Impossible to implement in interactive systems where required CPU time is not known.
- The processer should know in advance how much time process will take.

Given: Table of processes, and their Arrival time, Execution time

Process	Arrival Time	Execution Time	Service Time
P0	0	5	0
P1	1	3	5
P2	2	8	14
P3	3	6	8

Process	Arrival Time	Execute Time	Service Time
P0	0	5	3
P1	1	3	0
P2	2	8	16
P3	3	6	8



Waiting time of each process is as follows −

Process	Waiting Time
P0	0 - 0 = 0
P1	5 - 1 = 4
P2	14 - 2 = 12
P3	8 - 3 = 5

Average Wait Time: (0 + 4 + 12 + 5)/4 = 21 / 4 = 5.25

Examples for Shortest Job First

1. Process ID Arrival Time Burst Time
2. _
3. P0 1 3
4. P1 2 6
5. P2 1 2
6. P3 3 7
7. P4 2 4
8. P5 5 5

Now, we are going to solve this problem in both pre emptive and non pre emptive way

We will first solve the problem in non pre emptive way.

In Non pre emptive way, the process is executed until it is completed.

Non Pre Emptive Shortest Job First CPU Scheduling

Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time TAT = CT - AT	Waiting Time WT = CT - BT
P0	1	3	5	4	1
P1	2	6	20	18	12
P2	0	2	2	2	0
P3	3	7	27	24	17
P4	2	4	9	7	4
P5	5	5	14	10	5

Gantt Chart:

Now let us find out Average Completion Time, Average Turn Around Time, Average Waiting Time.

Average Completion Time:

1. Average Completion Time = (5 + 20 + 2 + 27 + 9 + 14) / 6
2. Average Completion Time = 77/6
3. Average Completion Time = 12.833

Average Waiting Time:

1. Average Waiting Time = (1 + 12 + 17 + 0 + 5 + 4) / 6
2. Average Waiting Time = 37 / 6

3. Average Waiting Time = 6.666

Average Turn Around Time:

1. Average Turn Around Time = (4 +18 + 2 +24 + 7 + 10) / 6
2. Average Turn Around Time = 65 / 6
3. Average Turn Around Time = 6.166

Pre Emptive Approach

In Pre emptive way, the process is executed when the best possible solution is found.

Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time TAT = CT - AT	Waiting Time WT = CT - BT
P0	1	3	5	4	1
P1	2	6	17	15	9
P2	0	2	2	2	0
P3	3	7	24	21	14
P4	2	4	11	9	5
P5	6	2	8	2	0

Gantt chart:

After P4 P5 is executed just because of the Pre Emptive Condition.

Now let us find out Average Completion Time, Average Turn Around Time, Average Waiting Time.

Average Completion Time

1. Average Completion Time = (5 + 17 + 2 + 24 + 11 +8) / 6
2. Average Completion Time = 67 / 6

3. Average Completion = 11.166

Average Turn Around Time

1. Average Turn Around Time = (4 +15 + 2 + 21 + 9 + 2) / 6
2. Average Turn Around Time = 53 / 6
3. Average Turn Around Time = 8.833

Average Waiting Time

1. Average Waiting Time = (1 + 9 + 0 + 14 + 5 + 0) /6
2. Average Waiting Time = 29 / 6
3. Average Waiting Time = 4.833

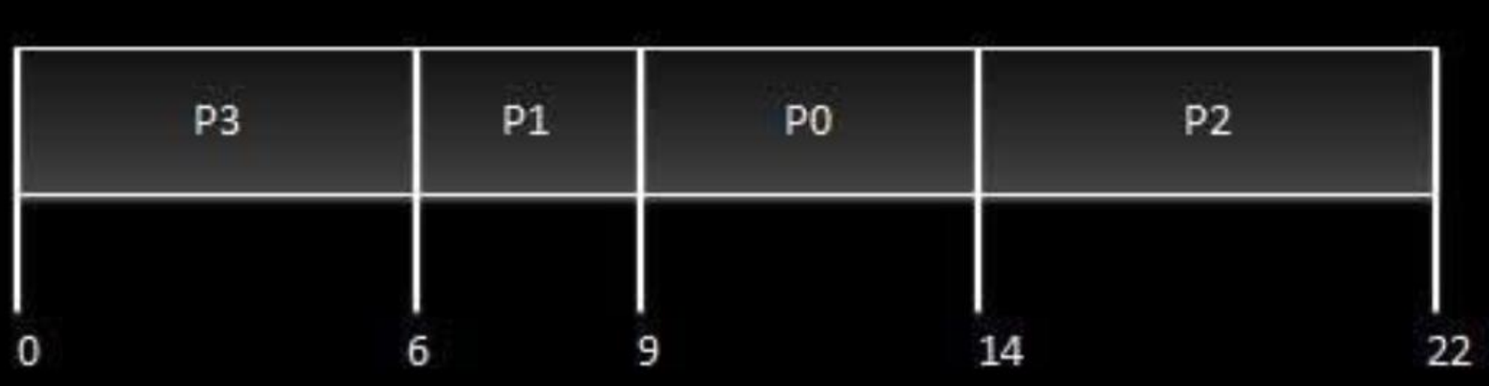
Priority Based Scheduling

- Priority scheduling is a non-preemptive algorithm and one of the most common scheduling algorithms in batch systems.
- Each process is assigned a priority. Process with highest priority is to be executed first and so on.
- Processes with same priority are executed on first come first served basis.
- Priority can be decided based on memory requirements, time requirements or any other resource requirement.

Given: Table of processes, and their Arrival time, Execution time, and priority. Here we are considering 1 is the lowest priority.

Process	Arrival Time	Execution Time	Priority	Service Time
P0	0	5	1	0
P1	1	3	2	11
P2	2	8	1	14
P3	3	6	3	5

Process	Arrival Time	Execute Time	Priority	Service Time
P0	0	5	1	9
P1	1	3	2	6
P2	2	8	1	14
P3	3	6	3	0



Waiting time of each process is as follows −

Process	Waiting Time
P0	0 - 0 = 0
P1	11 - 1 = 10
P2	14 - 2 = 12
P3	5 - 3 = 2

Average Wait Time: (0 + 10 + 12 + 2)/4 = 24 / 4 = 6

Examples:

Now, let us explain this problem with the help of an example of Priority Scheduling

1. S. No Process ID Arrival Time Burst Time Priority
2. _
3. 1 P 1 0 5 5
4. 2 P 2 1 6 4
5. 3 P 3 2 2 0
6. 4 P 4 3 1 2
7. 5 P 5 4 7 1
8. 6 P 6 4 6 3

Here, in this problem the priority number with highest number is least prioritized.

This means 5 has the least priority and 0 has the highest priority.

Solution:

S. No	Process Id	Arrival Time	Burst Time	Priority	Completion Time Turn Around Time TAT = CT - AT	Waiting Time WT = TAT - BT	
1	PP 1	P0	P5	P5	P5	P5	P0
2	PP 2	P1	P6	P4	P27	P26	P20
3	PP 3	P2	P2	P0	P7	P5	P3
4	PP 4	P3	P1	P2	P15	P12	P11
5	PP 5	P4	P7	P1	P14	P10	P3
6	P 6	P4	P6	P3	P21	P17	P11

Gantt Chart:

Average Completion Time

1. Average Completion Time = (5 +27 +7 +15 +14 + 21) / 6
2. Average Completion Time = 89 / 6
3. Average Completion Time = 14.8333

Average Waiting Time

1. Average Waiting Time = (0 + 20 + 3 + 11 + 3 + 11) / 6
2. Average Waiting Time = 48 / 6
3. Average Waiting Time = 7

Average Turn Around Time

1. Average Turn Around Time = (5 + 26 + 5 + 11 + 10 + 17) / 6

2. Average Turn Around Time = 74 / 6
3. Average Turn Around Time = 12.333

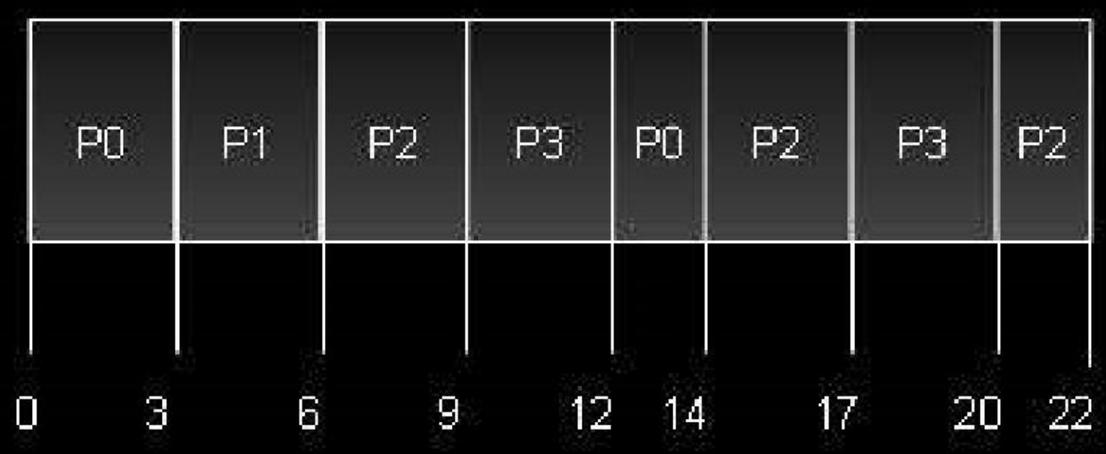
Shortest Remaining Time

- Shortest remaining time (SRT) is the preemptive version of the SJN algorithm.
- The processor is allocated to the job closest to completion but it can be preempted by a newer ready job with shorter time to completion.
- Impossible to implement in interactive systems where required CPU time is not known.
- It is often used in batch environments where short jobs need to give preference.

Round Robin Scheduling

- Round Robin is the preemptive process scheduling algorithm.
- Each process is provided a fix time to execute, it is called a **quantum**.
- Once a process is executed for a given time period, it is preempted and other process executes for a given time period.
- Context switching is used to save states of preempted processes.

Quantum = 3



Wait time of each process is as follows −

Process	Wait Time : Service Time - Arrival Time
P0	(0 - 0) + (12 - 3) = 9
P1	(3 - 1) = 2
P2	(6 - 2) + (14 - 9) + (20 - 17) = 12

P3 (9 - 3) + (17 - 12) = 11

Average Wait Time: (9+2+12+11) / 4 = 8.5

Examples:

Problem

Process ID	Arrival Time	Burst Time
P0	1	3
P1	0	5
P2	3	2
P3	4	3
P4	2	1

Solution:

Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
P0	1	3	5	4	1
P1	0	5	14	14	9
P2	3	2	7	4	2
P3	4	3	10	6	3
P4	2	1	3	1	0

Gantt Chart:

Average Completion Time = 7.8

Average Turn Around Time = 4.8

Average Waiting Time = 3

Multiple-Level Queues Scheduling

Multiple-level queues are not an independent scheduling algorithm. They make use of other existing algorithms to group and schedule jobs with common characteristics.

- Multiple queues are maintained for processes with common characteristics.
- Each queue can have its own scheduling algorithms.
- Priorities are assigned to each queue.

For example, CPU-bound jobs can be scheduled in one queue and all I/O-bound jobs in another queue. The Process Scheduler then alternately selects jobs from each queue and assigns them to the CPU based on the algorithm assigned to the queue.

Algorithm Evaluation

Evaluation of Process Scheduling Algorithms

The first thing we need to decide is how we will evaluate the algorithms. To do this we need to decide on the relative importance of the factors we listed above (Fairness, Efficiency, Response Times, Turnaround and Throughput). Only once we have decided on our evaluation method can we carry out the evaluation.

Deterministic Modeling

This evaluation method takes a predetermined workload and evaluates each algorithm using that workload. Assume we are presented with the following processes, which all arrive at time zero.

Process	Burst Time
P1	9
P2	33
P3	2
P4	5
P5	14

Which of the following algorithms will perform best on this workload?
First Come First Served (FCFS), Non Preemptive Shortest Job First (SJF) and Round Robin (RR). Assume a

quantum of 8 milliseconds.
Before looking at the answers, try to calculate the figures for each algorithm.
The advantages of deterministic modeling is that it is exact and fast to compute. The disadvantage is that it is only applicable to the workload that you use to test. As an example, use the above workload but assume P1 only has a burst time of 8 milliseconds. What does this do to the average waiting time?
Of course, the workload might be typical and scale up but generally deterministic modeling is too specific and requires too much knowledge about the workload.

Queuing Models

Another method of evaluating scheduling algorithms is to use queuing theory. Using data from real processes we can arrive at a probability distribution for the length of a burst time and the I/O times for a process. We can now generate these times with a certain distribution.
We can also generate arrival times for processes (arrival time distribution).
If we define a queue for the CPU and a queue for each I/O device we can test the various scheduling algorithms using queuing theory.
Knowing the arrival rates and the service rates we can calculate various figures such as average queue length, average wait time, CPU utilization etc.
One useful formula is ***Little's Formula.***

$n = \lambda w$

Where

n is the average queue length
λ is the average arrival rate for new processes (e.g. five a second)
w is the average waiting time in the queue

Knowing two of these values we can, obviously, calculate the third. For example, if we know that eight processes arrive every second and there are normally sixteen processes in the queue we can compute that the average waiting time per process is two seconds.
The main disadvantage of using queuing models is that it is not always easy to define realistic distribution times and we have to make assumptions. This results in the model only being an approximation of what actually happens.

Simulations

Rather than using queuing models we simulate a computer. A Variable, representing a clock is incremented. At each increment the state of the simulation is updated.
Statistics are gathered at each clock tick so that the system performance can be analysed.
The data to drive the simulation can be generated in the same way as the queuing model, although this leads to similar problems.
Alternatively, we can use trace data. This is data collected from real processes on real machines and is fed into the simulation. This can often provide good results and good comparisons over a range of scheduling algorithms.
However, simulations can take a long time to run, can take a long time to implement and the trace data may be difficult to collect and require large amounts of storage.

Implementation

The best way to compare algorithms is to implement them on real machines. This will give the best results but does have a number of disadvantages.

· It is expensive as the algorithm has to be written and then implemented on real hardware.
· If typical workloads are to be monitored, the scheduling algorithm must be used in a live situation. Users may not be happy with an environment that is constantly changing.
· If we find a scheduling algorithm that performs well there is no guarantee that this state will continue if the workload or environment changes.

Multiple Processors Scheduling in Operating System

Multiple processor scheduling or multiprocessor scheduling focuses on designing the system's scheduling function, which consists of more than one processor. Multiple CPUs share the load (load sharing) in multiprocessor scheduling so that various processes run simultaneously. In general, multiprocessor scheduling is complex as compared to single processor scheduling. In the multiprocessor scheduling, there are many processors, and they are identical, and we can run any process at any time.

The multiple CPUs in the system are in close communication, which shares a common bus, memory, and other peripheral devices. So we can say that the system is tightly coupled. These systems are used when we want to process a bulk amount of data, and these systems are mainly used in satellite, weather forecasting, etc.

There are cases when the processors are identical, i.e., homogenous, in terms of their functionality in multiple-processor scheduling. We can use any processor available to run any process in the queue.

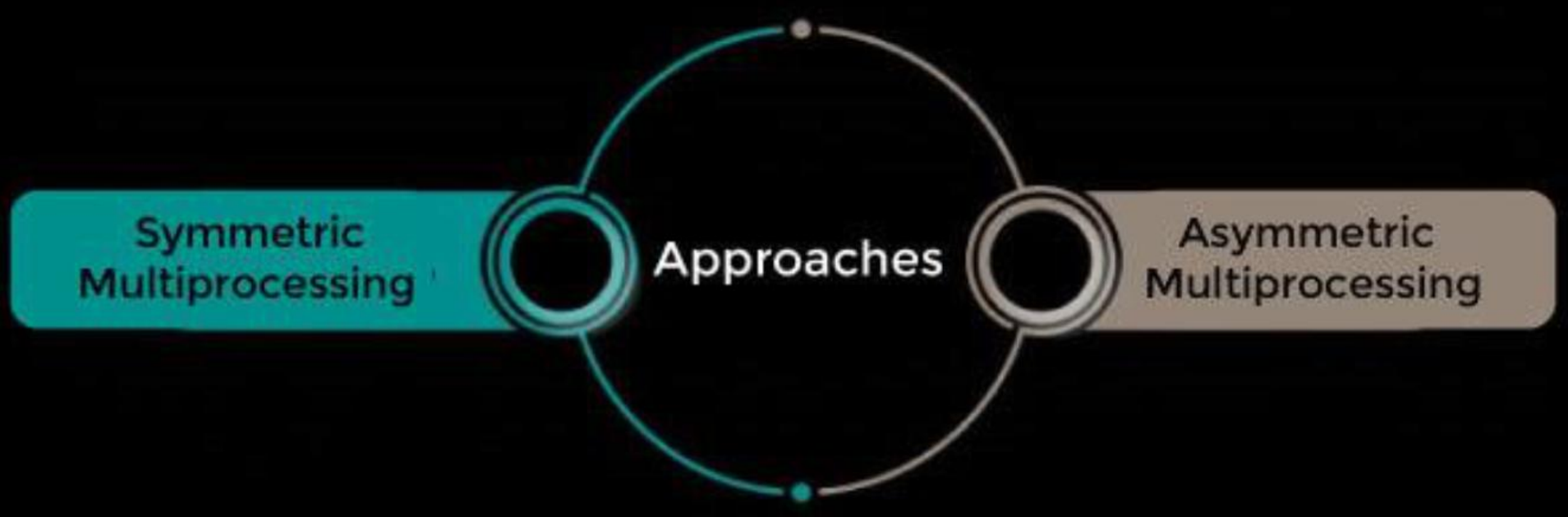
Multiprocessor systems may be ***heterogeneous*** (different kinds of CPUs) or ***homogenous*** (the same CPU). There may be special scheduling constraints, such as devices connected via a private bus to only one

CPU.

There is no policy or rule which can be declared as the best scheduling solution to a system with a single processor. Similarly, there is no best scheduling solution for a system with multiple processors as well.

Approaches to Multiple Processor Scheduling

There are two approaches to multiple processor scheduling in the operating system: Symmetric Multiprocessing and Asymmetric Multiprocessing.

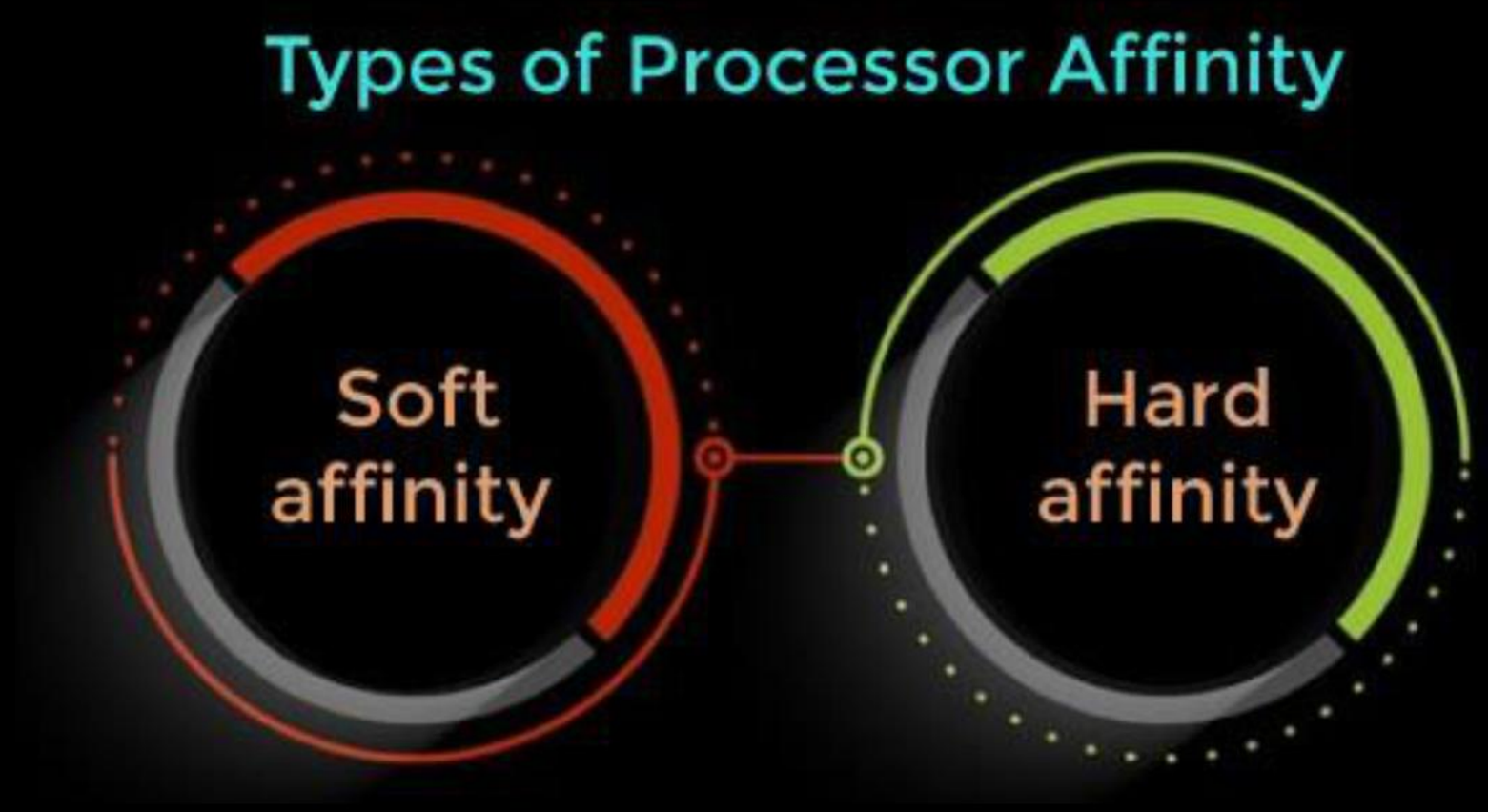


1. **Symmetric Multiprocessing:** It is used where each processor is ***self-scheduling***. All processes may be in a common ready queue, or each processor may have its private queue for ready processes. The scheduling proceeds further by having the scheduler for each processor examine the ready queue and select a process to execute.
2. **Asymmetric Multiprocessing:** It is used when all the scheduling decisions and I/O processing are handled by a single processor called the ***Master Server***. The other processors execute only the ***user code***. This is simple and reduces the need for data sharing, and this entire scenario is called Asymmetric Multiprocessing.

Processor Affinity

Processor Affinity means a process has an ***affinity*** for the processor on which it is currently running. When a process runs on a specific processor, there are certain effects on the cache memory. The data most recently accessed by the process populate the cache for the processor. As a result, successive memory access by the process is often satisfied in the cache memory.

Now, suppose the process migrates to another processor. In that case, the contents of the cache memory must be invalidated for the first processor, and the cache for the second processor must be repopulated. Because of the high cost of invalidating and repopulating caches, most SMP(symmetric multiprocessing) systems try to avoid migrating processes from one processor to another and keep a process running on the same processor. This is known as processor affinity. There are two types of processor affinity, such as:

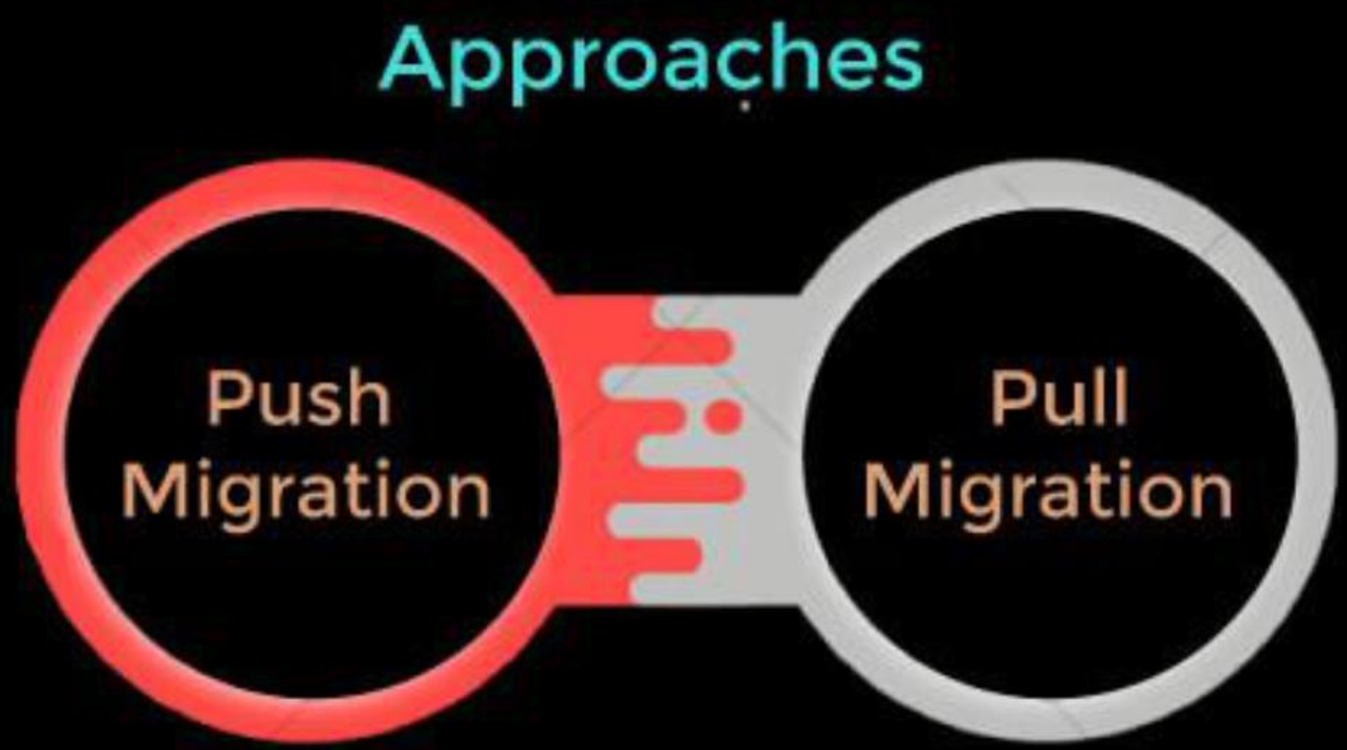


1. **Soft Affinity:** When an operating system has a policy of keeping a process running on the same processor but not guaranteeing it will do so, this situation is called soft affinity.
2. **Hard Affinity:** Hard Affinity allows a process to specify a subset of processors on which it may run. Some Linux systems implement soft affinity and provide system calls like ***sched_setaffinity()*** that also support hard affinity.

Load Balancing

Load Balancing is the phenomenon that keeps the workload evenly distributed across all processors in an SMP system. Load balancing is necessary only on systems where each processor has its own private queue of a process that is eligible to execute.

Load balancing is unnecessary because it immediately extracts a runnable process from the common run queue once a processor becomes idle. On SMP (symmetric multiprocessing), it is important to keep the workload balanced among all processors to utilize the benefits of having more than one processor fully. One or more processors will sit idle while other processors have high workloads along with lists of processors awaiting the CPU. There are two general approaches to load balancing:



1. **Push Migration:** In push migration, a task routinely checks the load on each processor. If it finds an imbalance, it evenly distributes the load on each processor by moving the processes from overloaded to idle or less busy processors.
2. **Pull Migration:**Pull Migration occurs when an idle processor pulls a waiting task from a busy processor for its execution.

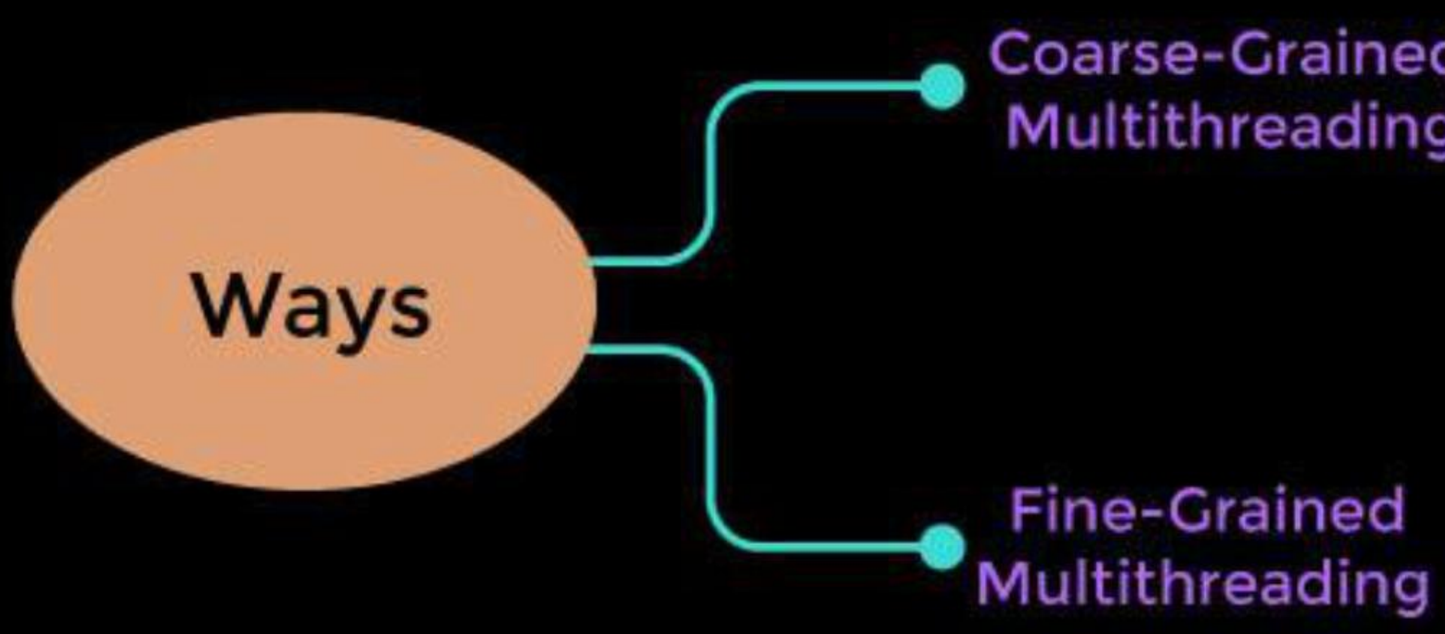
Multi-core Processors

In multi-core processors, multiple processor cores are placed on the same physical chip. Each core has a register set to maintain its architectural state and thus appears to the operating system as a separate physical processor. ***SMP systems*** that use multi-core processors are faster and consume less power than systems in which each processor has its own physical chip.

However, multi-core processors may complicate the scheduling problems. When the processor accesses memory, it spends a significant amount of time waiting for the data to become available. This situation is called a ***Memory stall***. It occurs for various reasons, such as cache miss, which is accessing the data that is not in the cache memory.

In such cases, the processor can spend upto 50% of its time waiting for data to become available from memory. To solve this problem, recent hardware designs have implemented

multithreaded processor cores in which two or more hardware threads are assigned to each core. Therefore if one thread stalls while waiting for the memory, the core can switch to another thread. There are two ways to multithread a processor:



1. **Coarse-Grained Multithreading:** A thread executes on a processor until a long latency event such as a memory stall occurs in coarse-grained multithreading. Because of the delay caused by the long latency event, the processor must switch to another thread to begin execution. The cost of switching between threads is high as the instruction pipeline must be terminated before the other thread can begin execution on the processor core. Once this new thread begins execution, it begins filling the pipeline with its instructions.
2. **Fine-Grained Multithreading:** This multithreading switches between threads at a much finer level, mainly at the boundary of an instruction cycle. The architectural design of fine-grained systems includes logic for thread switching, and as a result, the cost of switching between threads is small.

Symmetric Multiprocessor

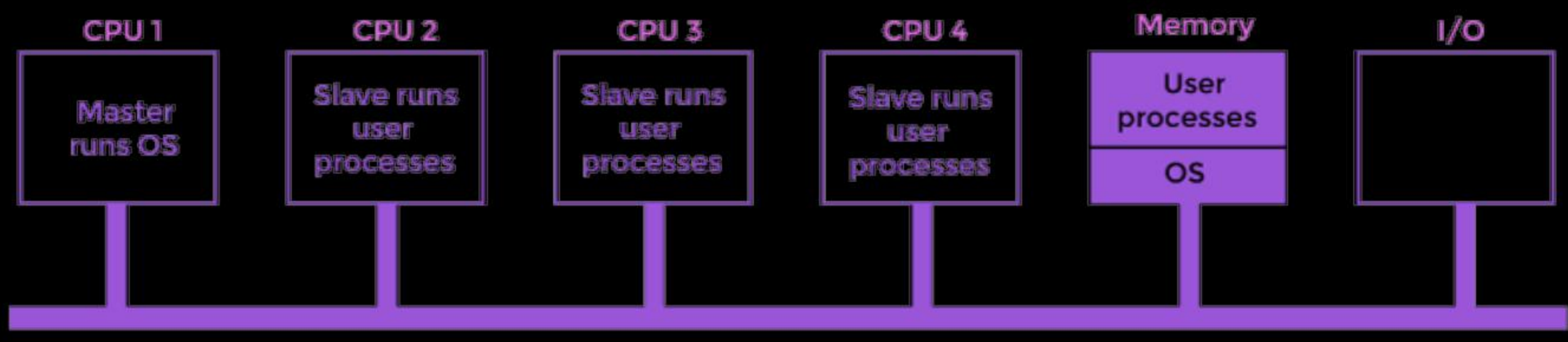
Symmetric Multiprocessors (SMP) is the third model. There is one copy of the OS in memory in this model, but any central processing unit can run it. Now, when a system call is made, the central processing unit on which the system call was made traps the kernel and processed that system call. This model balances processes and memory dynamically. This approach uses Symmetric Multiprocessing, where each processor is self-scheduling.

The scheduling proceeds further by having the scheduler for each processor examine the ready queue and select a process to execute. In this system, this is possible that all the process may be in a common ready queue or each processor may have its private queue for the ready process. There are mainly three sources of contention that can be found in a multiprocessor operating system.

- **Locking system:** As we know that the resources are shared in the multiprocessor system, there is a need to protect these resources for safe access among the multiple processors. The main purpose of the locking scheme is to serialize access of the resources by the multiple processors.
- **Shared data:** When the multiple processors access the same data at the same time, then there may be a chance of inconsistency of data, so to protect this, we have to use some protocols or locking schemes.
- **Cache coherence:** It is the shared resource data that is stored in multiple local caches. Suppose two clients have a cached copy of memory and one client change the memory block. The other client could be left with an invalid cache without notification of the change, so this conflict can be resolved by maintaining a coherent view of the data.

Master-Slave Multiprocessor

In this multiprocessor model, there is a single data structure that keeps track of the ready processes. In this model, one central processing unit works as a master and another as a slave. All the processors are handled by a single processor, which is called the master server.



The master server runs the operating system process, and the slave server runs the user processes. The memory and input-output devices are shared among all the processors, and all the processors are connected to a common bus. This system is simple and reduces data sharing, so this system is called ***Asymmetric multiprocessing***.

Virtualization and Threading

In this type of ***multiple processor*** scheduling, even a single CPU system acts as a multiple processor system. In a system with virtualization, the virtualization presents one or more virtual CPUs to each of the virtual machines running on the system. It then schedules the use of physical CPUs among the virtual machines.

- Most virtualized environments have one host operating system and many guest operating systems, and the host operating system creates and manages the virtual machines.
- Each virtual machine has a guest operating system installed, and applications run within that guest.

- Each guest operating system may be assigned for specific use cases, applications, or users, including time-sharing or real-time operation.
- Any guest operating-system scheduling algorithm that assumes a certain amount of progress in a given amount of time will be negatively impacted by the virtualization.
- A time-sharing operating system tries to allot 100 milliseconds to each time slice to give users a reasonable response time. A given 100 millisecond time slice may take much more than 100 milliseconds of virtual CPU time. Depending on how busy the system is, the time slice may take a second or more, which results in a very poor response time for users logged into that virtual machine.
- The net effect of such scheduling layering is that individual virtualized operating systems receive only a portion of the available CPU cycles, even though they believe they are receiving all cycles and scheduling all of those cycles. The time-of-day clocks in virtual machines are often incorrect because timers take no longer to trigger than they would on dedicated CPUs.
- Virtualizations can thus undo the good scheduling algorithm efforts of the operating systems within virtual machines.

Scheduling in Real Time Systems

- **Read**
- Discuss
- Courses

Real-time systems are systems that carry real-time tasks. These tasks need to be performed immediately with a certain degree of urgency. In particular, these tasks are related to control of certain events (or) reacting to them. Real-time tasks can be classified as hard real-time tasks and soft real-time tasks.
A hard real-time task must be performed at a specified time which could otherwise lead to huge losses. In soft real-time tasks, a specified deadline can be missed. This is because the task can be rescheduled (or) can be completed after the specified time,

In real-time systems, the scheduler is considered as the most important component which is typically a short-term task scheduler. The main focus of this scheduler is to reduce the response time associated with each of the associated processes instead of handling the deadline.

If a preemptive scheduler is used, the real-time task needs to wait until its corresponding tasks time slice completes. In the case of a non-preemptive scheduler, even if the highest priority is allocated to the task, it needs to wait

until the completion of the current task. This task can be slow (or) of the lower priority and can lead to a longer wait.

A better approach is designed by combining both preemptive and non-preemptive scheduling. This can be done by introducing time-based interrupts in priority based systems which means the currently running process is interrupted on a time-based interval and if a higher priority process is present in a ready queue, it is executed by preempting the current process.

Based on schedulability, implementation (static or dynamic), and the result (self or dependent) of analysis, the scheduling algorithm are classified as follows.

1. **Static table-driven approaches:**
   These algorithms usually perform a static analysis associated with scheduling and capture the schedules that are advantageous. This helps in providing a schedule that can point out a task with which the execution must be started at run time.

2. **Static priority-driven preemptive approaches:**
   Similar to the first approach, these type of algorithms also uses static analysis of scheduling. The difference is that instead of selecting a particular schedule, it provides a useful way of assigning priorities among various tasks in preemptive scheduling.

3. **Dynamic planning-based approaches:**
   Here, the feasible schedules are identified dynamically (at run time). It carries a certain fixed time interval and a process is executed if and only if satisfies the time constraint.

4. **Dynamic best effort approaches:**
   These types of approaches consider deadlines instead of feasible schedules. Therefore the task is aborted if its deadline is reached. This approach is used widely is most of the real-time systems.

Advantages of Scheduling in Real-Time Systems:

- **Meeting Timing Constraints:** Scheduling ensures that real-time tasks are executed within their specified timing constraints. It guarantees that critical tasks are completed on time, preventing potential system failures or losses.
- **Resource Optimization:** Scheduling algorithms allocate system resources effectively, ensuring efficient utilization of processor time, memory, and other resources. This helps maximize system throughput and performance.

- **Priority-Based Execution:** Scheduling allows for priority-based execution, where higher-priority tasks are given precedence over lower-priority tasks. This ensures that time-critical tasks are promptly executed, leading to improved system responsiveness and reliability.
- **Predictability and Determinism:** Real-time scheduling provides predictability and determinism in task execution. It enables developers to analyze and guarantee the worst-case execution time and response time of tasks, ensuring that critical deadlines are met.
- **Control Over Task Execution:** Scheduling algorithms allow developers to have fine-grained control over how tasks are executed, such as specifying task priorities, deadlines, and inter-task dependencies. This control facilitates the design and implementation of complex real-time systems.

Disadvantages of Scheduling in Real-Time Systems:

- **Increased Complexity:** Real-time scheduling introduces additional complexity to system design and implementation. Developers need to carefully analyze task requirements, define priorities, and select suitable scheduling algorithms. This complexity can lead to increased development time and effort.
- **Overhead:** Scheduling introduces some overhead in terms of context switching, task prioritization, and scheduling decisions. This overhead can impact system performance, especially in cases where frequent context switches or complex scheduling algorithms are employed.
- **Limited Resources:** Real-time systems often operate under resource-constrained environments. Scheduling tasks within these limitations can be challenging, as the available resources may not be sufficient to meet all timing constraints or execute all tasks simultaneously.
- **Verification and Validation:** Validating the correctness of real-time schedules and ensuring that all tasks meet their deadlines require rigorous testing and verification techniques. Verifying timing constraints and guaranteeing the absence of timing errors can be a complex and time-consuming process.
- **Scalability:** Scheduling algorithms that work well for smaller systems may not scale effectively to larger, more complex real-time systems. As the number of tasks and system complexity increases, scheduling decisions become more challenging and may require more advanced algorithms or approaches.